

파머완 6장 - 차원 축소

01. 차원 축소 개요

차원 축소 : 매우 많은 피처로 구성된 다차원 데이터 세트의 차원을 축소해 새로운 차원의 데이터 세트를 생성하는 것.

⇒ 시각적으로 데이터 압축 표현 가능, 학습 데이터의 크기가 줄어들

- 피처 선택 (feature selection) : 특정 피처에 종속성이 강한 불필요한 피처는 아예 제거하고, 데이터의 특징을 잘 나타내는 주요 피처만 선택하는 것
- 피처(특성) 추출 (feature extraction) : 기존 피처를 저차원의 중요 피처로 압축해서 추출하는 것 → 새롭게 추출된 중요 특성은 기존의 피처와는 완전히 다른 값이 됨 → 피처를 함축적으로 더 잘 설명할 수 있는 또다른 공간으로 매핑해 추출하는 것

대표적 차원 축소 알고리즘 : PCA, SVD, NMF

- 매우 많은 픽셀로 이뤄진 이미지 데이터에서 잠재된 특성을 피처로 도출해 함축적 형태의 이미지 변환과 압축을 수행 → 과적합 영향력이 적어져서 예측 성능 높아질 수 있음
- 텍스트 문서의 숨겨진 의미를 추출 : 문서 내 단어들의 구성에서 숨겨져 있는 시맨틱 (Semantic) 의미나 토픽(Topic)을 잠재 요소로 간주하고 이를 찾아낼 수 있음

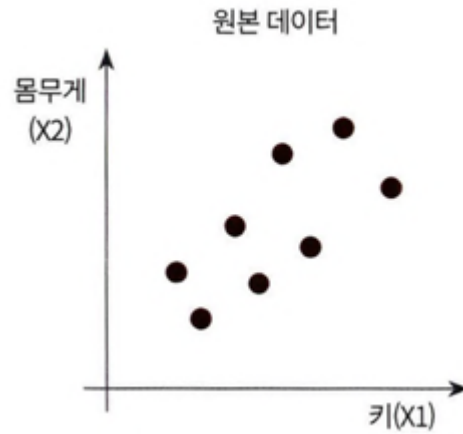
02. PCA (Principal Component Analysis)

[PCA]

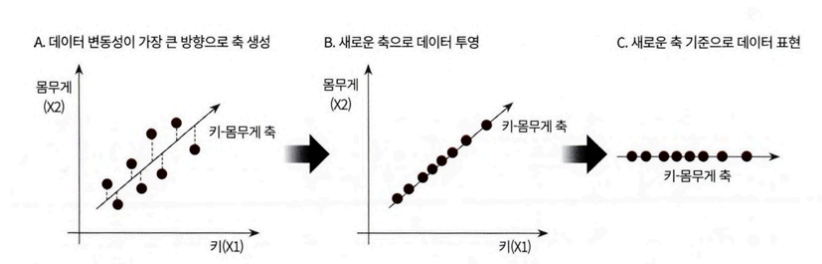
가장 대표적인 차원 축소 기법으로, 여러 변수 간에 존재하는 상관관계를 이용해 이를 대표하는 주성분(Principal Component)을 추출해 차원을 축소

- 데이터 정보 유실이 최소화하기 위해 PCA는 가장 높은 분산을 가지는 데이터의 축을 찾아 이 축으로 차원을 축소함 → 이것이 PCA의 주성분이 됨

ex) 키와 몸무게 2개의 피처를 가지고 있는 데이터 세트

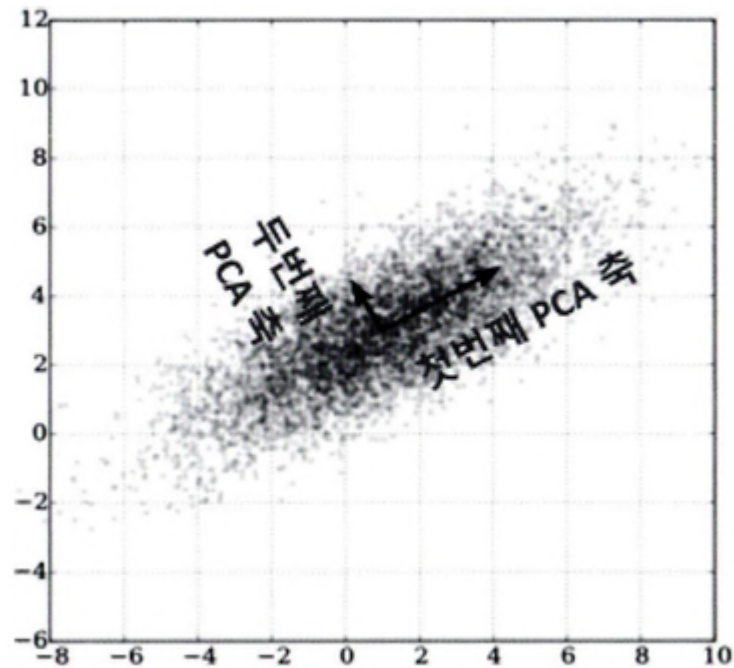


2개의 피처를 한 개의 주성분을 가진 데이터 세트로 차원 축소 : 데이터 변동성이 가장 큰 방향으로 축을 생성하고, 새롭게 생성된 축으로 데이터를 투영하는 방식



가장 큰 데이터 변동성(Variance)을 기반으로 첫 번째 벡터 축을 생성 → 두 번째 축은 이 벡터 축에 직각이 되는 벡터(직교 벡터)를 축으로 함 → 세 번째 축은 다시 두 번째 축과 직각이 되는 벡터를 설정하는 방식으로 축을 생성

⇒ 이렇게 생성된 벡터 축에 원본 데이터를 투영하면
벡터 축의 개수만큼의 차원으로 원본 데이터가 차원 축소됨



→ PCA (주성분 분석) : 원본 데이터의 피쳐 개수에 비해 매우 작은 주성분으로 원본 데이터의 총 변동성을 대부분 설명할 수 있는 분석법

[선형대수 관점의 해석] : 입력 데이터의 공분산 행렬을 고유값 분해하고, 구한 고유벡터 (PCA의 주성분 벡터)에 입력 데이터를 선형 변환하는 것

- 고유 벡터 : 행렬 A를 곱하더라도 방향이 변하지 않고 그 크기만 변하는 벡터
- 공분산 행렬: 정방행렬(Square Matrix, 열과 행 동일)이며 대칭행렬(Symmetric Matrix, 대각 원소를 중심으로 원소 값이 대칭)
- P : n*n 직교행렬, 시그마: n*n 정방행렬, P의 T는 행렬 P의 전치 행렬

$$C = P \Sigma P^T$$

$$C = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$$

→ 공분산 c는 고유벡터 직교 행렬 * 고유값 정방 행렬 * 고유벡터 직교 행렬의 전치 행렬로 분해

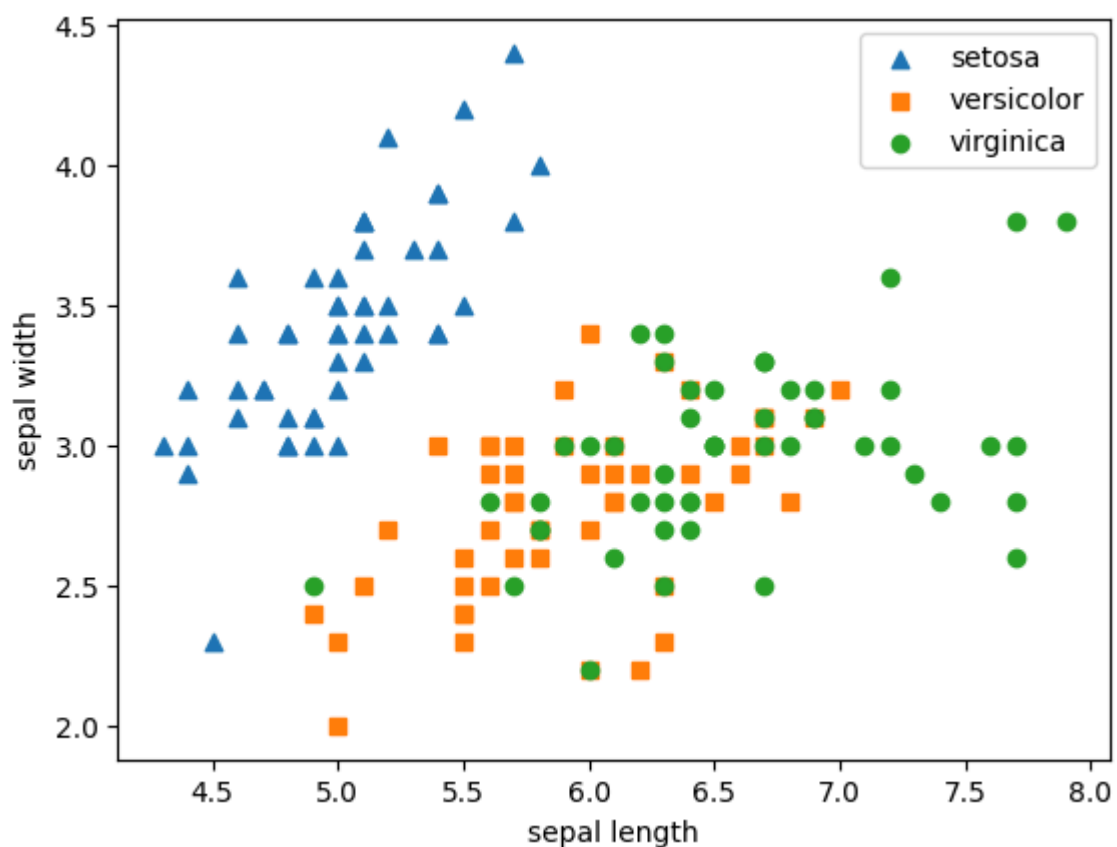
입력 데이터의 공분산 행렬이 고유벡터와 고유값으로 분해될 수 있으며, 이렇게 분해된 고유벡터를 이용해 입력 데이터를 선형 변환하는 방식이 PCA라는 것!

[PCA 단계]

1. 입력 데이터 세트의 공분산 행렬을 생성
2. 공분산 행렬의 고유벡터와 고유값을 계산
3. 고유값이 가장 큰 순으로 K개(PCA 변환 차수만큼)만큼 고유벡터를 추출
4. 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

[붓꽃 데이터 세트의 PCA]

- 각 품종에 따라 원본 붓꽃 데이터 세트가 어떻게 분포돼 있는지 2차원으로 시각화



- Standard Scaling 후 붓꽃 데이터를 2차원 데이터로 변환

- PCA 클래스는 생성 파라미터로 n_components를 입력받음, fit & transform 을 호출해 PCA 변환 수행

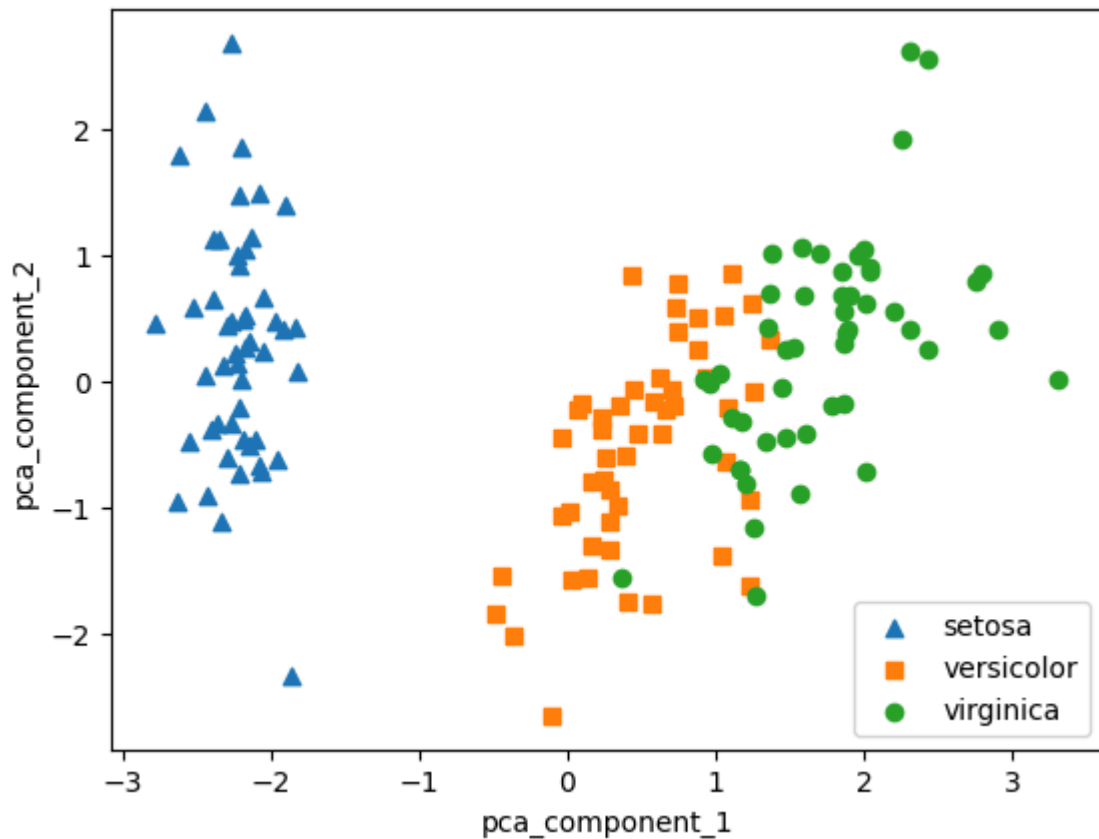
```
from sklearn.decomposition import PCA
pca = PCA(n_components = 2)
# fit()과 transform()을 호출해 PCA 변환 데이터 반환
pca.fit(iris_scaled)
iris_pca = pca.transform(iris_scaled)
print(iris_pca.shape)
```

- PCA 변환된 데이터 세트를 2차원상에서 시각화

```
markers=['^', 's', 'o']
# setosa를 세모, versicolor를 네모, virginica를 동그라미로 표시

# pca_component_1 을 x축, pc_component_2를 y축으로 scatter plot 수행
for i, marker in enumerate(markers):
    x_axis_data = irisDF_pca[irisDF_pca['target']==i]['pca_component_1']
    y_axis_data = irisDF_pca[irisDF_pca['target']==i]['pca_component_2']
    plt.scatter(x_axis_data, y_axis_data, marker=marker,label=iris.target_names[i])

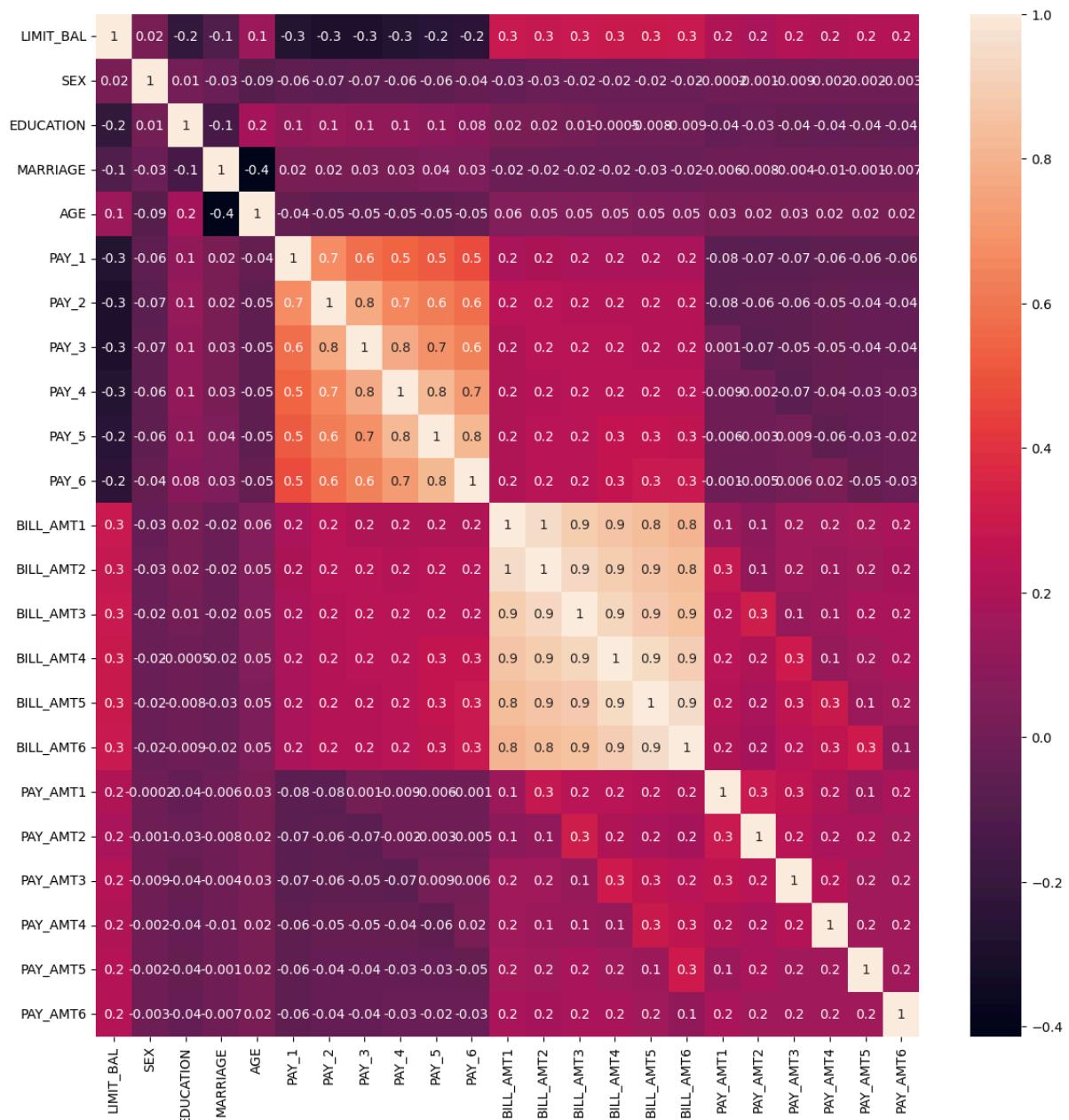
plt.legend()
plt.xlabel('pca_component_1')
plt.ylabel('pca_component_2')
plt.show()
```



- 첫 번째 PCA 변환 요소인 `pca_component_1`이 전체 변동성의 약 72.9%를 차지하며, 두 번째인 `pca_component_2`가 약 22.8% 차지 → PCA를 2개 요소로만 변환해도 원본 데이터의 변동성을 95% 설명 가능

[UCI Machine Learning Repository - 신용카드 고객 데이터 세트(Credit Card Clients Data Set)]

- 각 속성 간의 상관도를 구한 뒤 이를 시본(Seaborn)의 heatmap으로 시각화



→ BILL_AMT1 ~ BILL_AMT6 6개 속성끼리의 상관도가 대부분 0.9 이상으로 매우 높음 : 높은 상관도를 가진 속성들은 소수의 PCA만으로도 자연스럽게 이 속성들의 변동성을 수용할 수 있음

- 전체 23개 속성의 약 1/4 수준인 6개의 PCA 컴포넌트만으로도 원본 데이터를 기반으로 한 분류 예측결과보다 약 1~2% 정도의 예측 성능 저하만 발생 → 전체 속성의 1/4 정도만으로도 이 정도 수치의 예측 성능을 유지할 수 있다는 것은 PCA의 뛰어난 압축 능력을 잘 보여주는 것

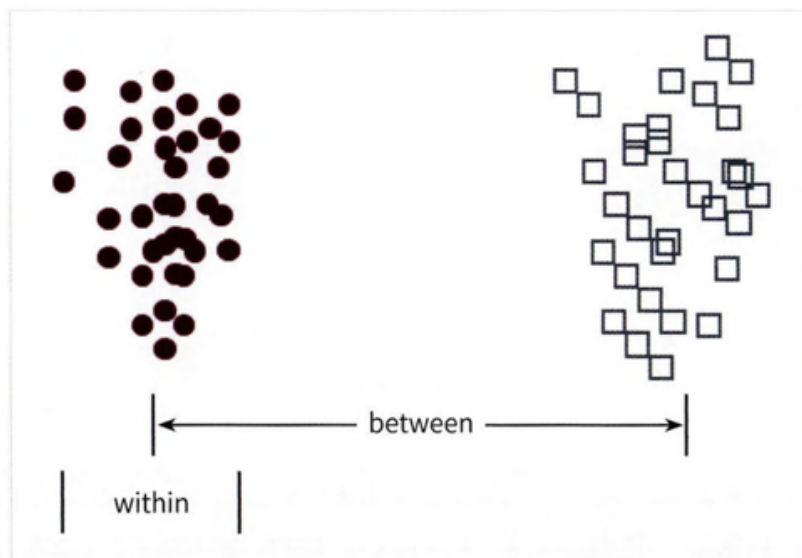
PCA는 컴퓨터 비전 분야에 활발하게 적용됨: 특히 얼굴 인식의 경우 Eigen-face라고 불리는 PCA 변환으로 원본 얼굴 이미지를 변환해 사용하는 경우가 많음

03. LDA (Linear Discriminant Analysis)

[LDA] 선형 판별 분석법 - PCA와 유사하게 입력 데이터 세트를 저차원 공간에 투영해 차원을 축소하는 기법

→ 차이점 : LDA는 지도학습의 분류(Classification)에서 사용하기 쉽도록 개별 클래스를 분별할 수 있는 기준을 최대한 유지하면서 차원을 축소함. PCA는 입력 데이터의 변동성의 가장 큰 축을 찾았지만, LDA는 입력 데이터의 결정 값 클래스를 최대한으로 분리할 수 있는 축을 찾음

- 클래스 간 분산, 클래스 내부 분산의 비율을 최대화하는 방식으로 차원 축소



- 공분산 행렬이 아니라 클래스 간 분산과 클래스 내부 분산 행렬을 생성한 뒤, 이 행렬에 기반해 고유벡터를 구하고 입력 데이터를 투영

LDA 구하는 방법

1. 클래스 내부와 클래스 간 분산 행렬을 구한다. 이 두 개의 행렬은 입력 데이터의 결정 값 클래스별로 개별 피처의 평균 벡터(mean vector)를 기반으로 구한다.
2. 클래스 내부 분산 행렬을 S 의 W , 클래스 간 분산 행렬을 S 의 B 라고 하면 다음 식으로 두 행렬을 고유벡터로 분해

$$S_W^T S_B = \begin{bmatrix} e_1 & \cdots & e_n \end{bmatrix} \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

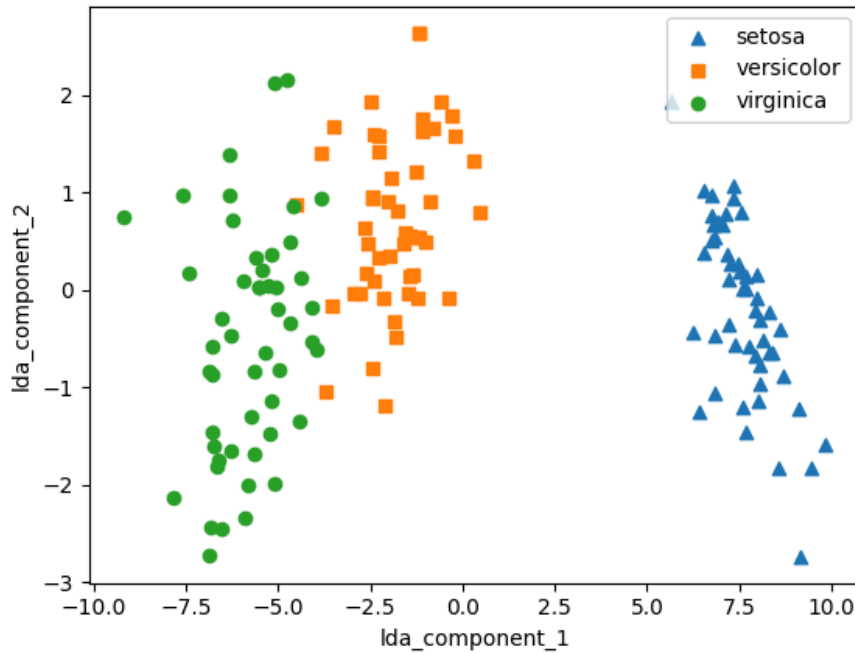
3. 고유값이 가장 큰 순으로 K개(LDA변환 차수만큼) 추출
4. 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

[붓꽃 데이터 세트에 LDA 적용하기]

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
lda_columns=['lda_component_1', 'lda_component_2']
irisDF_lda = pd.DataFrame(iris_lda, columns=lda_columns)
irisDF_lda['target']=iris.target
#setosa는 세모, versicolor는 네모, virginica는 동그라미로 표현
markers=['^','s', 'o']
#setosa의 target 값은 0, versicolor는 1, virginica는 2. 각 target별로 다른 모양으로
for i, marker in enumerate(markers):
    x_axis_data = irisDF_lda [irisDF_lda ['target']==i] ['lda_component_1']
    y_axis_data = irisDF_lda [irisDF_lda ['target']==i] ['lda_component_2']

    plt.scatter(x_axis_data, y_axis_data, marker=marker, label=iris.target_names[i])

plt.legend(loc='upper right')
plt.xlabel('lda_component_1')
plt.ylabel('lda_component_2')
plt.show()
```



04. SVD(Singular Value Decomposition)

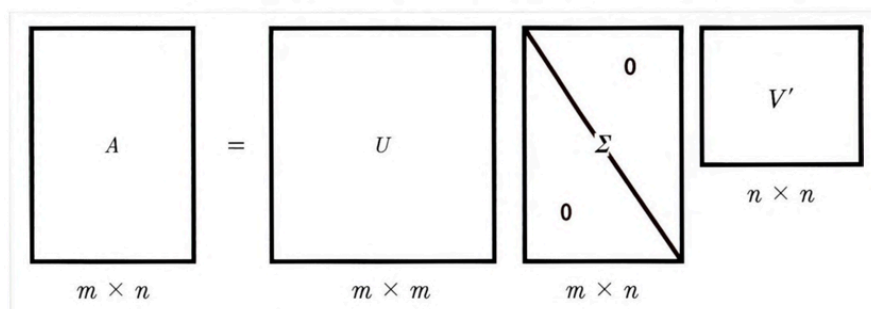
[SVD] 특이값 분해, 행과 열의 크기가 다른 행렬에도 적용

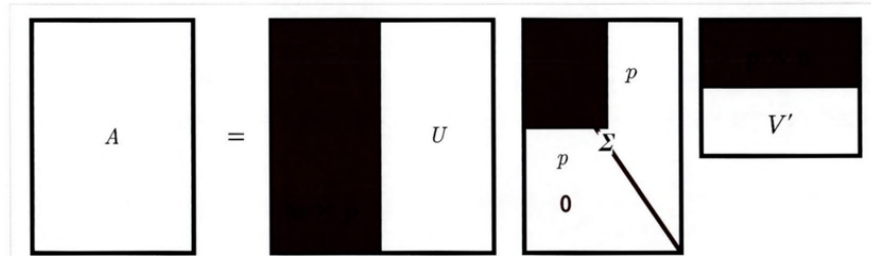
$$A = U \Sigma V^T$$

시그마는 대각행렬로, 행렬의 대각에 위치한 값만 0이 아니고 나머지 위치의 값은 모두 0

시그마가 위치한 0이 아닌 값이 바로 행렬 A의 특이값

→ SVD는 A의 차원이 $m \times n$ 일 때 U의 차원이 $m \times m$, 시그마의 차원이 $m \times n$, V의 T의 차원이 $n \times n$ 으로 분해함





Truncated SVD는 시그마의 대각원소 중에 상위 몇 개만 추출해서 여기에 대응하는 U, V의 원소도 함께 제거해 더욱 차원을 줄인 형태로 분해

```
# 넘파이의 svd 모듈 импорт
import numpy as np
from numpy.linalg import svd
```

```
np.random.seed(121)
a = np.random.randn(4, 4)
print(np.round(a, 3))
```

```
#SVD 적용
U, Sigma, Vt = svd(a)
print(U.shape, Sigma.shape, Vt.shape)
print('U matrix : \n', np.round(U, 3))
print('Sigma Value : \n', np.round(Sigma, 3))
print('V transpose matrix : \n', np.round(Vt, 3))
```

[결과]

(4, 4) (4,) (4, 4)

U matrix :

```
[[-0.079 -0.318  0.867  0.376]
 [ 0.383  0.787  0.12   0.469]
 [ 0.656  0.022  0.357 -0.664]
 [ 0.645 -0.529 -0.328  0.444]]
```

Sigma Value :

```
[3.423 2.023 0.463 0.079]
```

V transpose matrix :

```
[[ 0.041  0.224  0.786 -0.574]
 [-0.2   0.562  0.37  0.712]]
```

```
[-0.778  0.395 -0.333 -0.357]
[-0.593 -0.692  0.366  0.189]]
```

- Truncated SVD를 이용해 행렬을 분해 : Truncated SVD는 시그마 행렬에 있는 대각 원소, 즉 특이값 중 상위 일부 데이터만 추출해 분해하는 방식 → 원본 행렬을 정확하게 원복은 불가능

```
import numpy as np
from scipy.sparse.linalg import svds
from scipy.linalg import svd
# 원본 행렬을 출력하고 SVD를 적용할 경우 U, Sigma, Vt의 차원 확인
np.random.seed(121)
matrix = np.random.random((6, 6))
print('원본 행렬:\n', matrix)
U, Sigma, Vt = svd(matrix, full_matrices=False)
print('\n분해 행렬 차원:', U.shape, Sigma.shape, Vt.shape)
print('\nSigma값 행렬:', Sigma)
# Truncated SVD로 Sigma 행렬의 특이값을 4개로 하여 Truncated SVD 수행 .
num_components = 4
U_tr, Sigma_tr, Vt_tr = svds(matrix, k=num_components)
print('\nTruncated SVD 분해 행렬 차원:', U_tr.shape, Sigma_tr.shape, Vt_tr.shape)
print('\nTruncated SVD Sigma값 행렬:', Sigma_tr)
matrix_tr = np.dot(np.dot(U_tr, np.diag(Sigma_tr)), Vt_tr) # output of Truncated
print('\nTruncated SVD로 분해 후 복원 행렬:\n', matrix_tr)
```

결과

```

원본 행렬:
[[0.11133083 0.21076757 0.23296249 0.15194456 0.83017814 0.40791941]
 [0.5557906 0.74552394 0.24849976 0.9686594 0.95268418 0.48984885]
 [0.01829731 0.85760612 0.40493829 0.62247394 0.29537149 0.92958852]
 [0.4056155 0.56730065 0.24575605 0.22573721 0.03827786 0.58098021]
 [0.82925331 0.77326256 0.94693849 0.73632338 0.67328275 0.74517176]
 [0.51161442 0.46920965 0.6439515 0.82081228 0.14548493 0.01806415]]

분해 행렬 차원: (6, 6) (6,) (6, 6)

Sigma값 행렬: [3.2535007 0.88116505 0.83865238 0.55463089 0.35834824 0.0349925 ]

Truncated SVD 분해 행렬 차원: (6, 4) (4,) (4, 6)

Truncated SVD Sigma값 행렬: [0.55463089 0.83865238 0.88116505 3.2535007 ]

Truncated SVD로 분해 후 복원 행렬:
[[0.19222941 0.21792946 0.15951023 0.14084013 0.81641405 0.42533093]
 [0.44874275 0.72204422 0.34594106 0.99148577 0.96866325 0.4754868 ]
 [0.12656662 0.88860729 0.30625735 0.59517439 0.28036734 0.93961948]
 [0.23989012 0.51026588 0.39697353 0.27308905 0.05971563 0.57156395]
 [0.83806144 0.78847467 0.93868685 0.72673231 0.6740867 0.73812389]
 [0.59726589 0.47953891 0.56613544 0.80746028 0.13135039 0.03479656]]

```

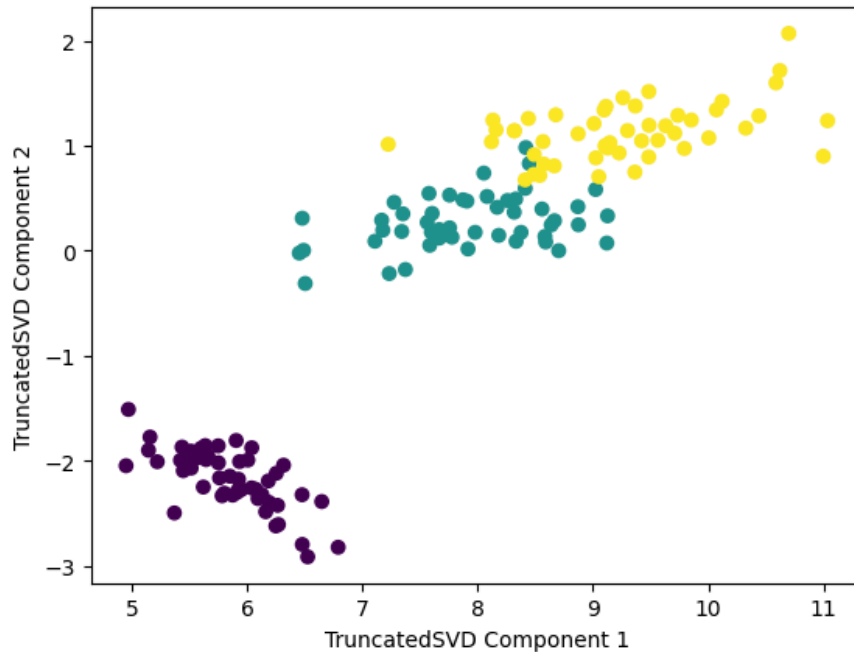
[사이킷런 TruncatedSVD 클래스를 이용한 변환]

- 사이킷런의 TruncatedSVD 클래스는 PCA 클래스와 유사하게 fit()와 transform()을 호출해 원본 데이터를 몇 개의 주요 컴포넌트(Truncated SVD의 K 컴포넌트 수)로 차원을 축소해 변환
- 원본 데이터를 Truncated SVD 방식으로 분해된 $U \times \text{Sigma}$ 행렬에 선형 변환해 생성

```

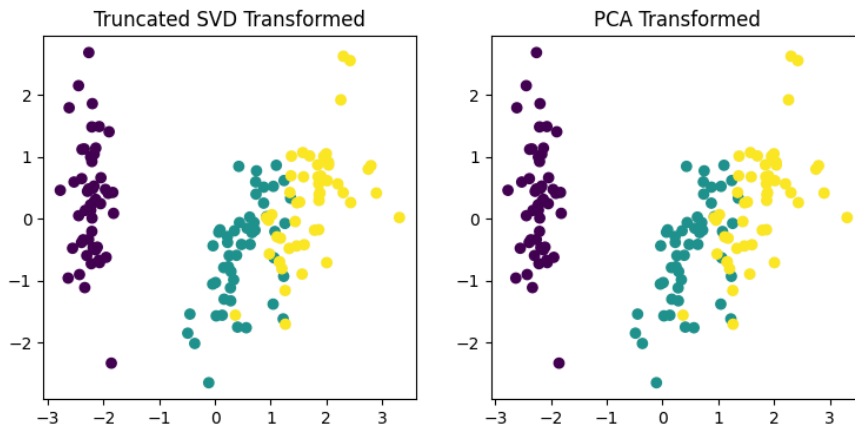
from sklearn.decomposition import TruncatedSVD, PCA
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
%matplotlib inline
iris = load_iris()
iris_fts = iris.data
# 2개의 주요 컴포넌트로 TruncatedSVD 변환
tsvd = TruncatedSVD(n_components=2)
tsvd.fit(iris_fts)
iris_tsvd = tsvd.transform(iris_fts)
# 산점도 2차원으로 TruncatedSVD 변환된 데이터 표현. 품종은 색깔로 구분
plt.scatter(x=iris_tsvd[:, 0], y= iris_tsvd[:,1], c= iris.target)
plt.xlabel('TruncatedSVD Component 1')
plt.ylabel('TruncatedSVD Component 2')

```



→ TruncatedSVD 변환 역시 PCA와 유사하게 변환 후에 품종별로 어느 정도 클러스터링이 가능할 정도로 각 변환 속성으로 뛰어난 고유성을 가지고 있음

```
from sklearn.preprocessing import StandardScaler
# 붓꽃 데이터를 StandardScaler로 변환
scaler = StandardScaler()
iris_scaled = scaler.fit_transform(iris_fts)
# 스케일링된 데이터를 기반으로 TruncatedSVD 변환 수행
tsvd = TruncatedSVD(n_components=2)
tsvd.fit(iris_scaled)
iris_tsvd = tsvd.transform(iris_scaled)
# 스케일링된 데이터를 기반으로 PCA 변환 수행
pca = PCA(n_components=2)
pca.fit(iris_scaled)
iris_pca = pca.transform(iris_scaled)
# TruncatedSVD 변환 데이터를 왼쪽에, PCA 변환 데이터를 오른쪽에 표현
fig, (ax1, ax2) = plt.subplots(figsize=(9, 4), ncols=2)
ax1.scatter(x=iris_tsvd[:,0], y= iris_tsvd[:, 1], c= iris.target)
ax2.scatter(x=iris_pca[:,0], y= iris_pca[:, 1], c= iris.target)
ax1.set_title('Truncated SVD Transformed')
ax2.set_title('PCA Transformed')
```



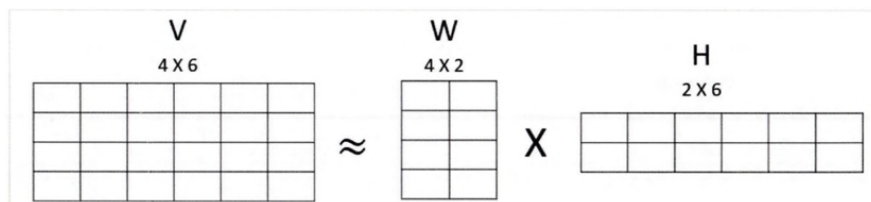
→ 두 개의 변환 행렬 값과 원본 속성별 컴포넌트 비율값을 실제로 서로 비교해 보면 거의 같음

- SVD는 PCA와 유사하게 컴퓨터 비전 영역에서 이미지 압축을 통한 패턴 인식과 신호 처리 분야에 사용됨

05. NMF(Non-Negative Matrix Factorization)

[NMF] Truncated SVD와 같이 낮은 랭크를 통한 행렬 근사 방식의

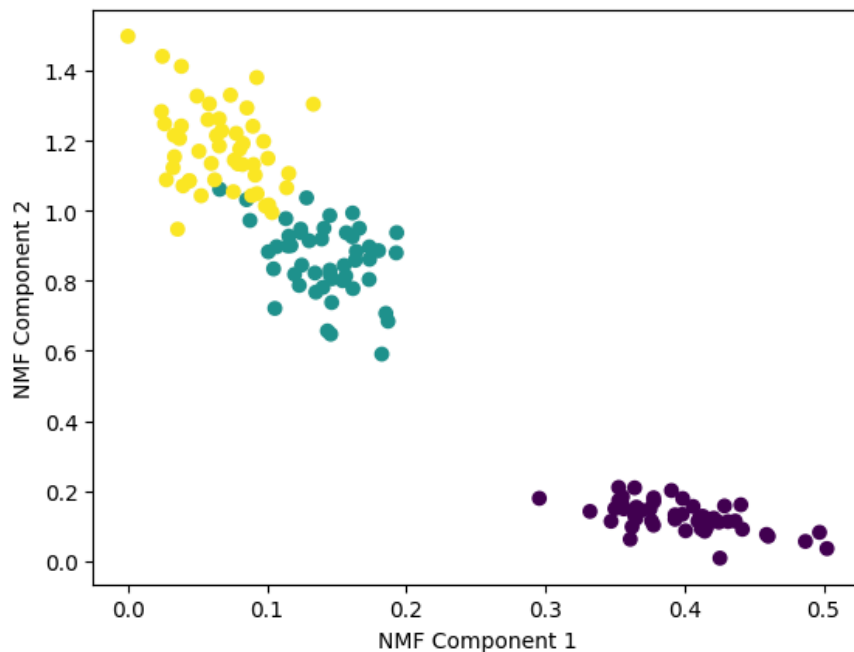
변형 → NMF는 원본 행렬 내의 모든 원소 값이 모두 양수(0 이상)라는 게 보장되면 다음과 같이 좀 더 간단하게 두 개의 기반 양수 행렬로 분해될 수 있는 기법을 지칭



- SVD와 유사하게 차원 축소를 통한 잠재 요소 도출로 이미지 변환 및 압축, 텍스트의 토픽 도출 등의 영역에서 사용
- 붓꽃 데이터를 NMF를 이용해 2개의 컴포넌트로 변환하고 이를 시각화

```
from sklearn.decomposition import NMF
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
%matplotlib inline
iris = load_iris()
iris_fts = iris.data
nmf = NMF(n_components=2)
```

```
nmf.fit(iris_fts)
iris_nmf = nmf.transform(iris_fts)
plt.scatter(x=iris_nmf[:, 0], y= iris_nmf[:, 1], c= iris.target)
plt.xlabel('NMF Component 1')
plt.ylabel('NMF Component 2')
```



→ NMF도 SVD와 유사하게 이미지 압축을 통한 패턴 인식, 텍스트의 토픽 모델링 기법, 문서 유사도 및 클러스터링에 잘 사용되며, 추천 영역에 활발하게 적용됨

06. 정리

- 차원 축소는 단순히 피처의 개수를 줄이는 개념보다는 이를 통해 데이터를 잘 설명할 수 있는 잠재적인 요소를 추출하는 것이 의미있음
- PCA는 입력 데이터의 변동성이 가장 큰 축을 구하고, 다시 이 축에 직각인 축을 반복적으로 축소하려는 차원 개수만큼 구한 뒤 입력 데이터를 이 축들에 투영해 차원을 축소하는 방식
- 입력 데이터의 공분산 행렬을 기반으로 고유 벡터(Eigenvector)를 생성하고 이렇게 구한 고유 벡터에 입력 데이터를 선형 변환
- LDA(Linear Discriminant Analysis)는 PCA와 매우 유사한 차원 축소 방식 : PCA가 입력 데이터 변동성의 가장 큰 축을 찾음 ↔ LDA는 입력 데이터의 결정 값 클래스를 최대한으로 분리할 수 있는 축을 찾는 방식으로 차원을 축소

- SVD와 NMF는 매우 많은 피쳐 데이터를 가진 고차원 행렬을 두 개의 저차원 행렬로 분리하는 행렬 분해 기법 : 특히 이러한 행렬 분해를 수행하면서 원본 행렬에서 잠재된 요소를 추출